MarkdownCheck

Casos de Prueba

Curso	Lenguajes Formales y de Programación
Proyecto	Proyecto 2 - Segundo Semestre 2026
Estudiante	Josué Eliú Castro Sosa
Carné	202405110
Sección	B+
Universidad	Universidad de San Carlos de Guatemala

Cómo se hicieron las pruebas

Cada caso es un archivo .md de la carpeta ejemplos. Se carga en la ventana de MarkdownCheck, se presiona Analizar y se revisan la tabla de tokens, la tabla de errores y si los reportes quedan habilitados. En cada caso se anota lo que debería pasar y lo que pasó de verdad, y se incluyen las capturas de la ventana tomadas durante la prueba.

Hay 13 casos: un documento válido completo, una lista de 3 niveles, citas anidadas, los 7 tipos de error del enunciado y 3 casos borde.

Resumen

No.	Archivo	Qué prueba	Resultado
1	ejemplo_valido.md	Documento válido con todos los elementos obligatorios	Correcto
2	lista_3_niveles.md	Lista con 3 niveles de anidamiento	Correcto
3	citas_anidadas.md	Citas anidadas en 2 niveles	Correcto
4	negrita_sin_cerrar.md	Error NEGRITA_SIN_CERRAR	Correcto
5	cursiva_sin_cerrar.md	Error CURSIVA_SIN_CERRAR	Correcto
6	codigo_inline_sin_cerrar.md	Error CODIGO_INLINE_SIN_CERRAR	Correcto
7	bloque_sin_cerrar.md	Error BLOQUE_CODIGO_SIN_CERRAR	Correcto
8	encabezado_invalido.md	Error ENCABEZADO_INVALIDO (más de 6 #)	Correcto
9	indentacion_invalida.md	Error INDENTACION_INVALIDA (salto de 0 a 4 espacios)	Correcto
10	cita_mal_formada.md	Error CITA_MAL_FORMADA (> sin espacio)	Correcto
11	borde_negrita_cursiva.md	Caso borde: cursiva dentro de negrita	Correcto
12	borde_sin_espacio.md	Caso borde: # sin espacio no es encabezado	Correcto
13	borde_vacio.md	Caso borde: archivo vacío	Correcto

Los 13 casos dieron el resultado esperado.

Caso 1: Documento válido con todos los elementos obligatorios

Archivo: `ejemplo_valido.md`

Entrada

````
# Proyecto de Lenguajes Formales

Este curso combina **teoría sólida** y *práctica de código*, con ejemplos en `Python`.

## Contenido del curso

- Análisis léxico
  - Autómatas finitos
  - Expresiones regulares
- Análisis sintáctico
  - Gramáticas libres de contexto
    - Recursión por la izquierda
    - Recursión por la derecha
- Generación de código

> El análisis sintáctico es donde la teoría se encuentra con la práctica.
>> Y donde más se aprende, según los estudiantes.

```python
def hola():
    print("Hola LFP")
```

---

Fin del documento.
````

Resultado esperado

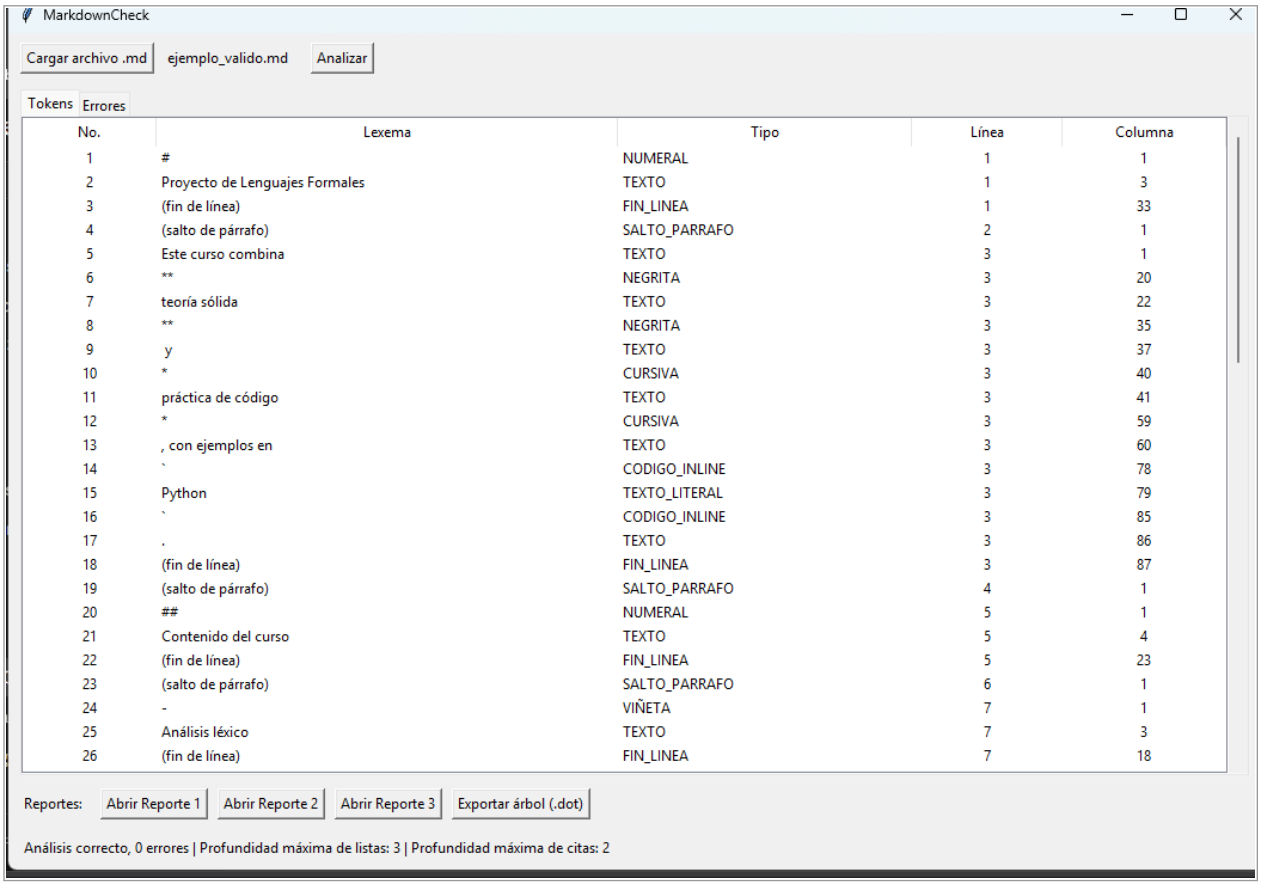
Sin errores. Debe reconocer encabezados H1 y H2, párrafo con negrita, cursiva y código en línea, lista de 3 niveles, cita de 2 niveles, bloque de código con lenguaje python y regla horizontal. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado obtenido

Sin errores. Tokens en orden: NUMERAL, TEXTO, FIN_LINEA, SALTO_PARRAFO, NEGRITA, CURSIVA, CODIGO_INLINE, TEXTO_LITERAL, INDENT, DEDENT, CITA CITA, CERCA_CODIGO, LENGUAJE, REGLA_HORIZONTAL, EOF. Profundidad máxima de listas: 3. Profundidad máxima de citas: 2. Estadísticas: 67 palabras, 2 párrafos, H1 = 1, H2 = 1, 1 bloque de código. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan. La captura muestra el inicio de la tabla de tokens (la tabla continúa con scroll), los botones de reportes habilitados y la barra de estado: «Análisis correcto, 0 errores | Profundidad máxima de listas: 3 | Profundidad máxima de citas: 2».

Resultado: **Correcto**

Captura



No.	Lexema	Tipo	Línea	Columna
1	#	NUMERAL	1	1
2	Proyecto de Lenguajes Formales	TEXTO	1	3
3	(fin de línea)	FIN_LINEA	1	33
4	(salto de párrafo)	SALTO_PARRAFO	2	1
5	Este curso combina	TEXTO	3	1
6	**	NEGRITA	3	20
7	teoría sólida	TEXTO	3	22
8	**	NEGRITA	3	35
9	y	TEXTO	3	37
10	*	CURSIVA	3	40
11	práctica de código	TEXTO	3	41
12	*	CURSIVA	3	59
13	, con ejemplos en	TEXTO	3	60
14	`	CODIGO_INLINE	3	78
15	Python	TEXTO_LITERAL	3	79
16	`	CODIGO_INLINE	3	85
17	.	TEXTO	3	86
18	(fin de línea)	FIN_LINEA	3	87
19	(salto de párrafo)	SALTO_PARRAFO	4	1
20	##	NUMERAL	5	1
21	Contenido del curso	TEXTO	5	4
22	(fin de línea)	FIN_LINEA	5	23
23	(salto de párrafo)	SALTO_PARRAFO	6	1
24	-	VIÑETA	7	1
25	Análisis léxico	TEXTO	7	3
26	(fin de línea)	FIN_LINEA	7	18

Caso 1: Pestaña Tokens, botones habilitados y barra de estado

Caso 2: Lista con 3 niveles de anidamiento

Archivo: `lista_3_niveles.md`

Entrada

```
- nivel cero
  - nivel uno
    - nivel dos
```

Resultado esperado

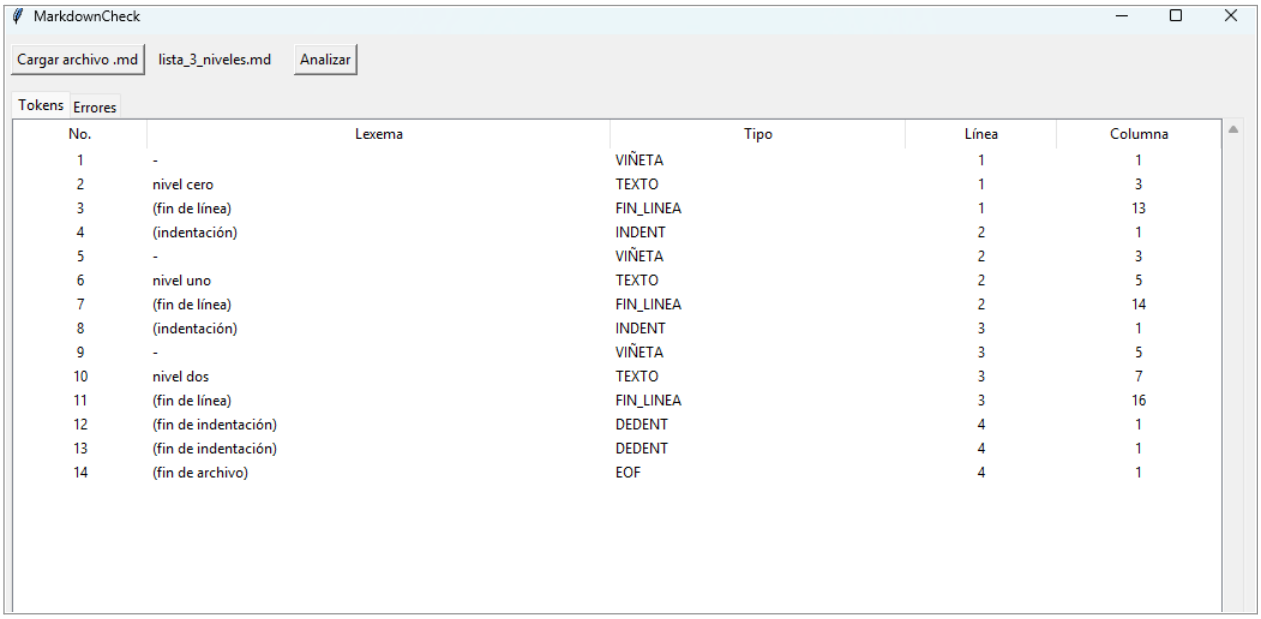
Sin errores. Debe generar un INDENT por cada nivel que se anida y dos DEDENT al final. Profundidad de listas 3.

Resultado obtenido

VIÑETA TEXTO('nivel cero') FIN_LINEA, INDENT, VIÑETA TEXTO('nivel uno') FIN_LINEA, INDENT, VIÑETA TEXTO('nivel dos') FIN_LINEA, DEDENT, DEDENT, EOF. Sin errores. Profundidad de listas 3. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado: **Correcto**

Captura



No.	Lexema	Tipo	Línea	Columna
1	-	VIÑETA	1	1
2	nivel cero	TEXTO	1	3
3	(fin de línea)	FIN_LINEA	1	13
4	(indentación)	INDENT	2	1
5	-	VIÑETA	2	3
6	nivel uno	TEXTO	2	5
7	(fin de línea)	FIN_LINEA	2	14
8	(indentación)	INDENT	3	1
9	-	VIÑETA	3	5
10	nivel dos	TEXTO	3	7
11	(fin de línea)	FIN_LINEA	3	16
12	(fin de indentación)	DEDENT	4	1
13	(fin de indentación)	DEDENT	4	1
14	(fin de archivo)	EOF	4	1

Caso 2: Pestaña Tokens

Caso 3: Citas anidadas en 2 niveles

Archivo: `citas_anidadas.md`

Entrada

```
> uno
>> dos
```

Resultado esperado

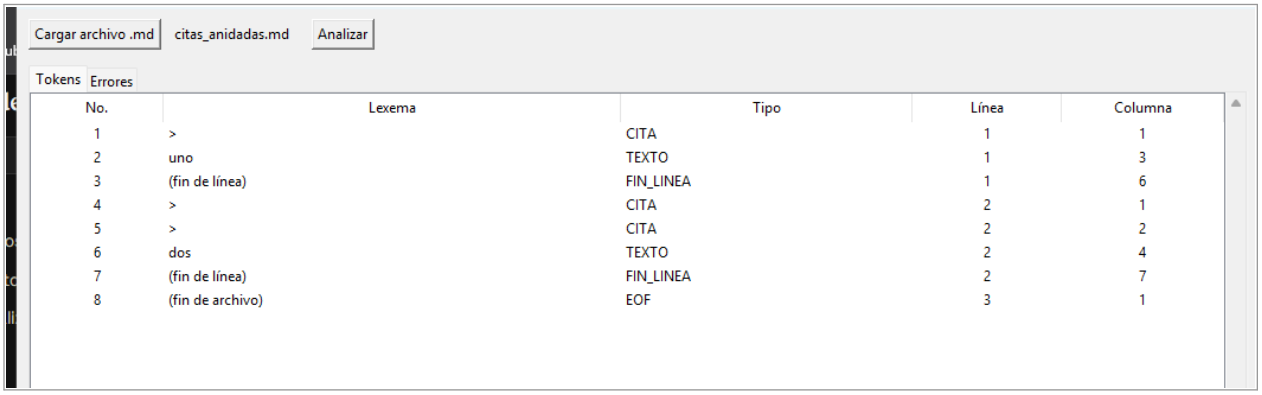
Sin errores. La segunda línea debe dar dos tokens CITA, uno por cada >. Profundidad de citas 2.

Resultado obtenido

CITA TEXTO('uno') FIN_LINEA, CITA CITA TEXTO('dos') FIN_LINEA, EOF. Sin errores. Profundidad de citas 2. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado: **Correcto**

Captura



No.	Lexema	Tipo	Línea	Columna
1	>	CITA	1	1
2	uno	TEXTO	1	3
3	(fin de línea)	FIN_LINEA	1	6
4	>	CITA	2	1
5	>	CITA	2	2
6	dos	TEXTO	2	4
7	(fin de línea)	FIN_LINEA	2	7
8	(fin de archivo)	EOF	3	1

Caso 3: Pestaña Tokens

Caso 4: Error NEGRITA_SIN_CERRAR

Archivo: `negrita_sin_cerrar.md`

Entrada

```
Esto es **importante
```

Resultado esperado

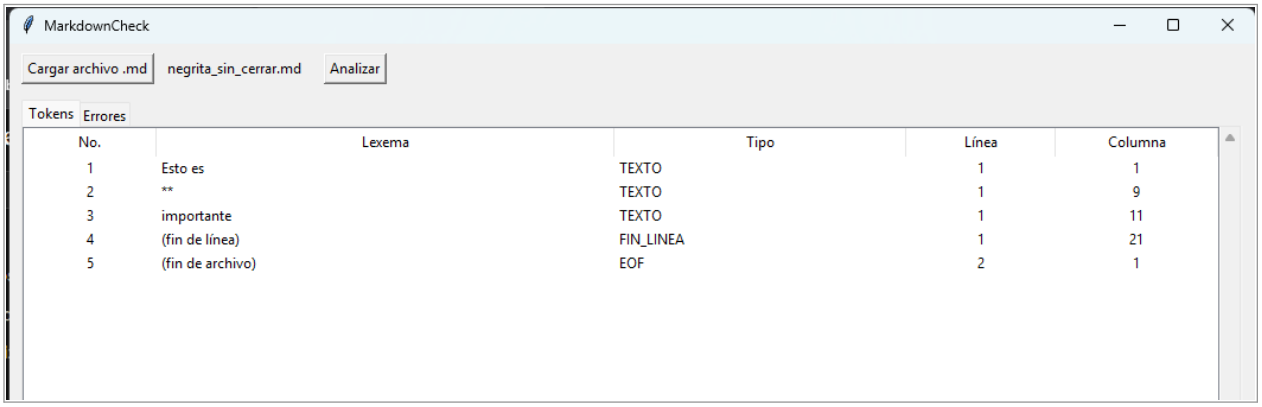
Un error NEGRITA_SIN_CERRAR en la posición donde se abrió (línea 1, columna 9). El análisis continúa.

Resultado obtenido

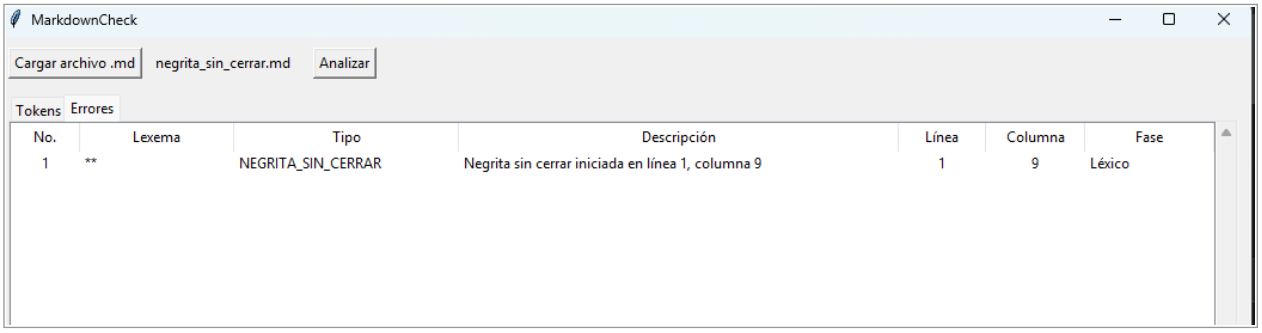
Tokens: TEXTO('Esto es '), TEXTO('**'), TEXTO('importante'), FIN_LINEA, EOF. Error: NEGRITA_SIN_CERRAR, línea 1, columna 9, fase Léxico. Mensaje: "Negrita sin cerrar iniciada en línea 1, columna 9". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 4: Pestaña Tokens



Caso 4: Pestaña Errores

Caso 5: Error CURSIVA_SIN_CERRAR

Archivo: `cursiva_sin_cerrar.md`

Entrada

```
Esto es *importante
```

Resultado esperado

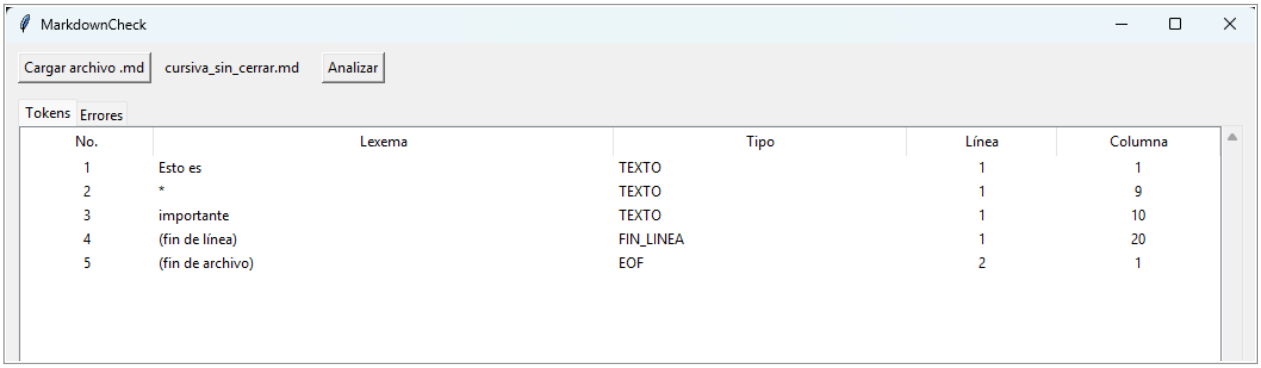
Un error CURSIVA_SIN_CERRAR en línea 1, columna 9. El análisis continúa.

Resultado obtenido

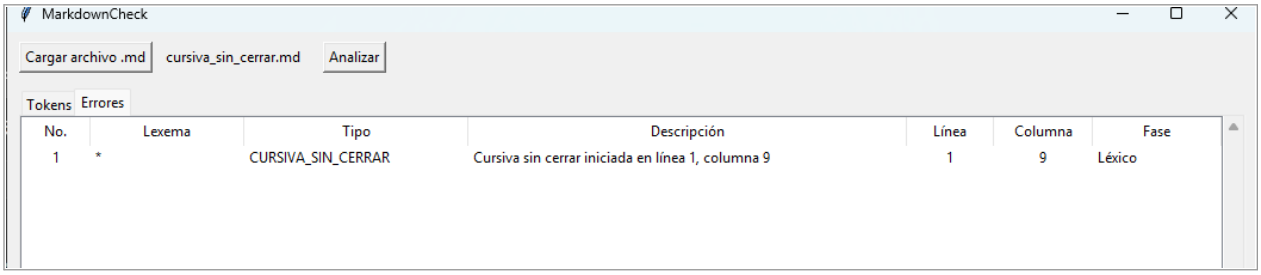
Tokens: TEXTO('Esto es '), TEXTO('*'), TEXTO('importante'), FIN_LINEA, EOF. Error: CURSIVA_SIN_CERRAR, línea 1, columna 9, fase Léxico. Mensaje: "Cursiva sin cerrar iniciada en línea 1, columna 9". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 5: Pestaña Tokens



Caso 5: Pestaña Errores

Caso 6: Error CODIGO_INLINE_SIN_CERRAR

Archivo: `codigo_inline_sin_cerrar.md`

Entrada

```
Esto es `abierto
```

Resultado esperado

Un error CODIGO_INLINE_SIN_CERRAR en línea 1, columna 9. El análisis continúa.

Resultado obtenido

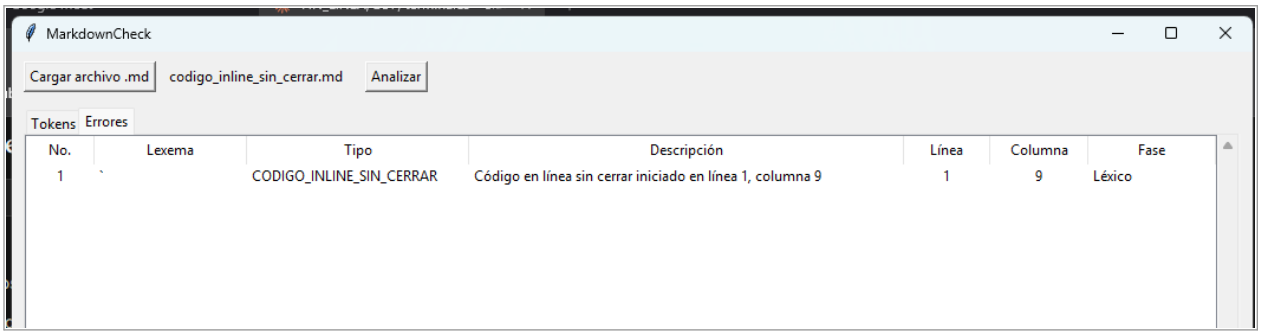
Tokens: TEXTO('Esto es `abierto'), FIN_LINEA, EOF. Error: CODIGO_INLINE_SIN_CERRAR, línea 1, columna 9, fase Léxico. Mensaje: "Código en línea sin cerrar iniciado en línea 1, columna 9". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 6: Pestaña Tokens



Caso 6: Pestaña Errores

Caso 7: Error BLOQUE_CODIGO_SIN_CERRAR

Archivo: `bloque_sin_cerrar.md`

Entrada

```
```python
print(1)
```

Resultado esperado

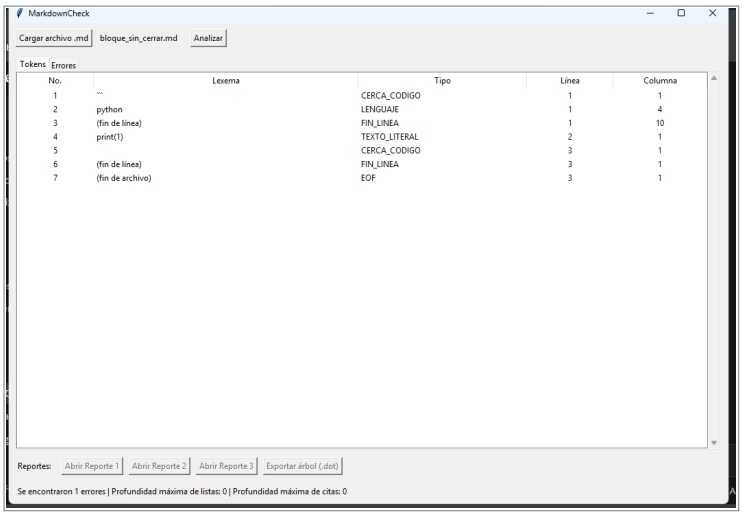
Un error BLOQUE_CODIGO_SIN_CERRAR en la línea donde se abrió la cerca (línea 1). El analizador cierra el bloque por su cuenta para poder seguir.

Resultado obtenido

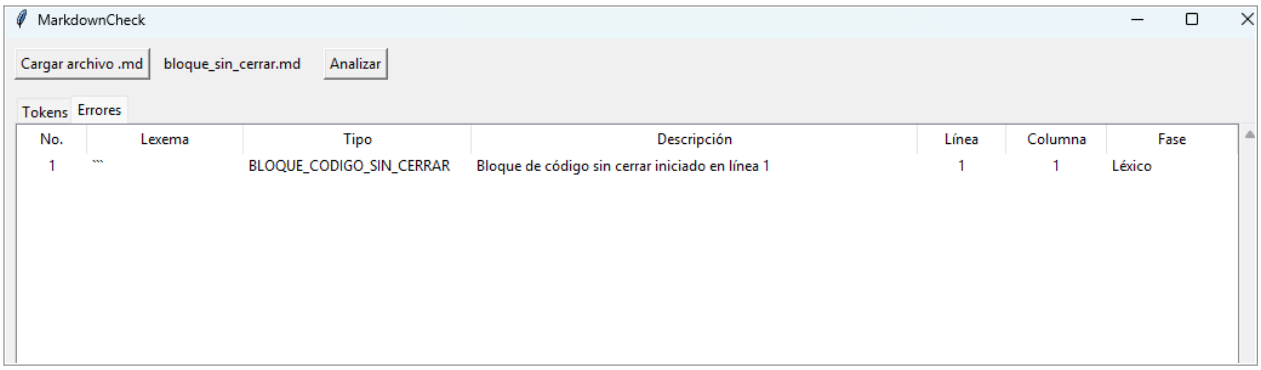
Tokens: CERCA_CODIGO, LENGUAJE('python'), FIN_LINEA, TEXTO_LITERAL('print(1)'), CERCA_CODIGO(''), FIN_LINEA, EOF. Error: BLOQUE_CODIGO_SIN_CERRAR, línea 1, columna 1, fase Léxico. Mensaje: "Bloque de código sin cerrar iniciado en línea 1". Un solo error, sin errores en cascada. No se generan reportes. Los botones de reportes quedan deshabilitados; la barra de estado indica «Se encontraron 1 errores».

Resultado: **Correcto**

Captura



Caso 7: Pestaña Tokens, botones deshabilitados y barra de estado



Caso 7: Pestaña Errores
```

Caso 8: Error ENCABEZADO_INVALIDO (más de 6 #)

Archivo: `encabezado_invalido.md`

Entrada

```
####### demasiado
```

Resultado esperado

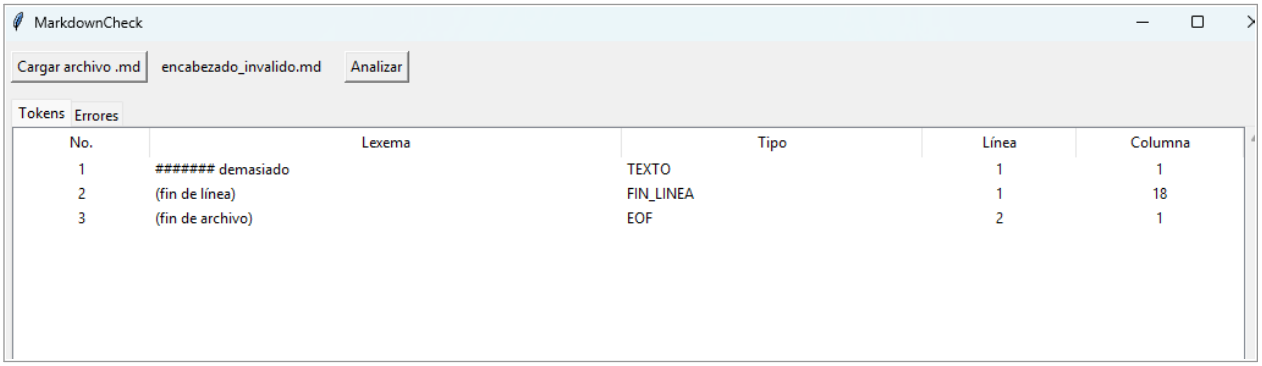
Un error ENCABEZADO_INVALIDO en línea 1 y la línea se trata como texto.

Resultado obtenido

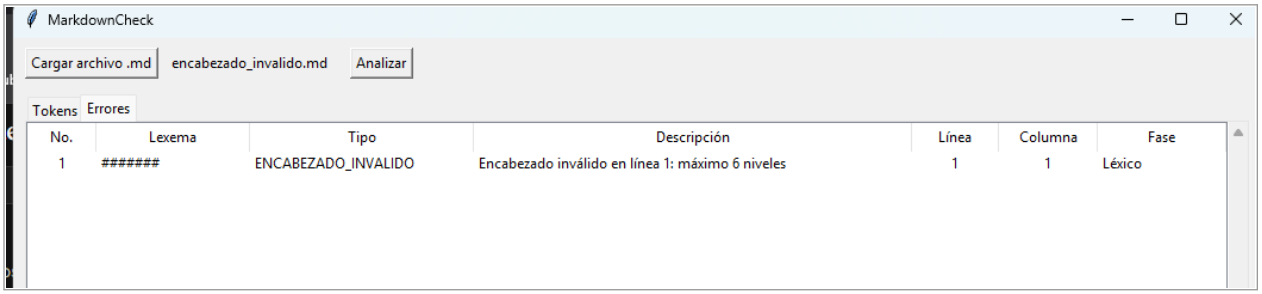
Tokens: TEXTO('####### demasiado'), FIN_LINEA, EOF. Error: ENCABEZADO_INVALIDO, línea 1, columna 1, fase Léxico. Mensaje: "Encabezado inválido en línea 1: máximo 6 niveles". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 8: Pestaña Tokens



Caso 8: Pestaña Errores

Caso 9: Error INDENTACION_INVALIDA (salto de 0 a 4 espacios)

Archivo: `indentacion_invalida.md`

Entrada

```
- uno
    - dos
```

Resultado esperado

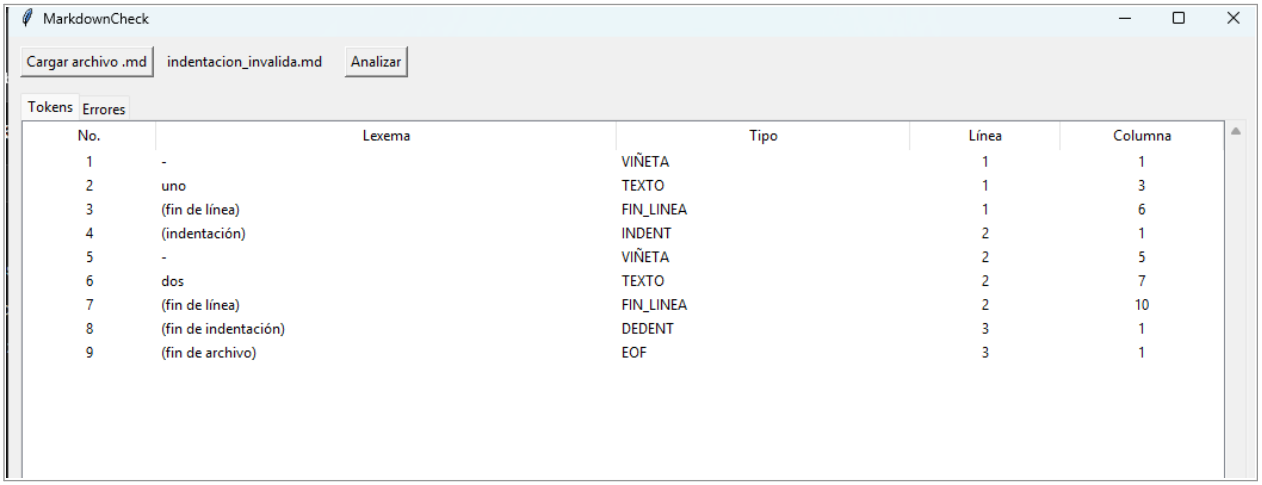
Un error INDENTACION_INVALIDA en la línea 2, porque la sangría sube dos niveles de una vez. El análisis continúa y el árbol se construye igual.

Resultado obtenido

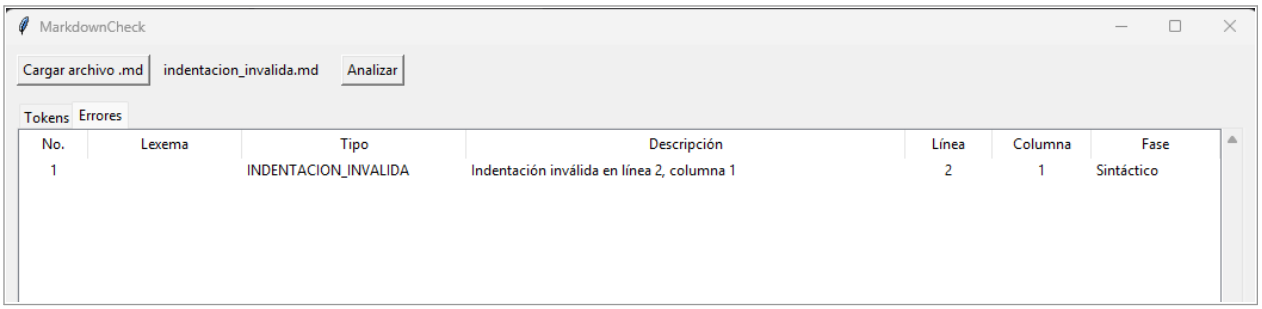
Tokens: VIÑETA TEXTO('uno') FIN_LINEA, INDENT, VIÑETA TEXTO('dos') FIN_LINEA, DEDENT, EOF. Error: INDENTACION_INVALIDA, línea 2, columna 1, fase Sintáctico. Mensaje: "Indentación inválida en línea 2, columna 1". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 9: Pestaña Tokens



Caso 9: Pestaña Errores

Caso 10: Error CITA_MAL_FORMADA (> sin espacio)

Archivo: `cita_mal_formada.md`

Entrada

```
>sin espacio
```

Resultado esperado

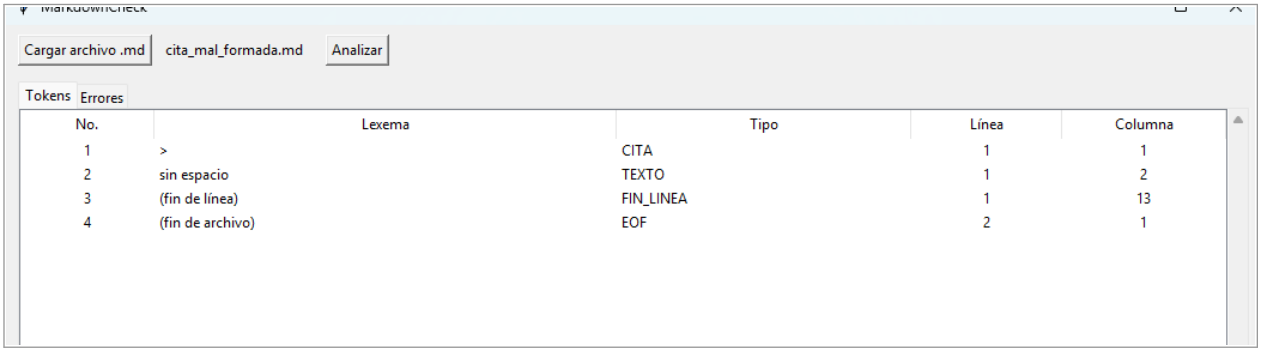
Un error CITA_MAL_FORMADA en línea 1, columna 1. El token CITA se emite igual.

Resultado obtenido

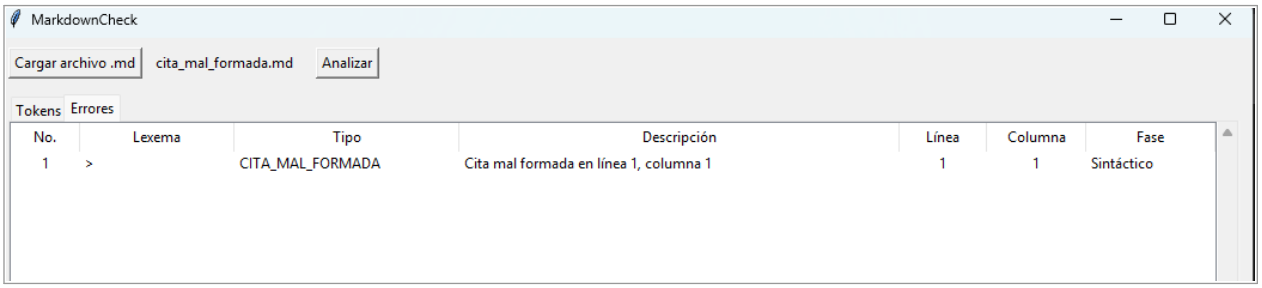
Tokens: CITA, TEXTO('sin espacio'), FIN_LINEA, EOF. Error: CITA_MAL_FORMADA, línea 1, columna 1, fase Sintáctico. Mensaje: "Cita mal formada en línea 1, columna 1". No se generan reportes. Los botones de reportes quedan deshabilitados.

Resultado: **Correcto**

Captura



Caso 10: Pestaña Tokens



Caso 10: Pestaña Errores

Caso 11: Caso borde: cursiva dentro de negrita

Archivo: `borde_negrita_cursiva.md`

Entrada

```
**negrita con *cursiva* adentro**
```

Resultado esperado

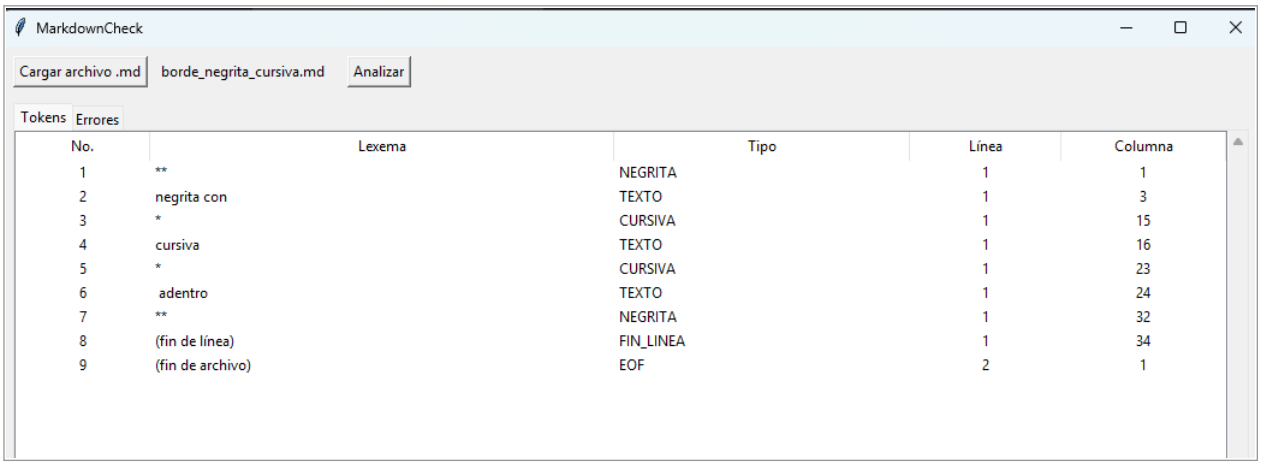
Sin errores. La cursiva queda dentro de la negrita, y en el HTML sale con adentro.

Resultado obtenido

Tokens: NEGRITA, TEXTO('negrita con '), CURSIVA, TEXTO('cursiva'), CURSIVA, TEXTO(' adentro'), NEGRITA, FIN_LINEA, EOF. Sin errores. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado: **Correcto**

Captura



No.	Lexema	Tipo	Línea	Columna
1	**	NEGRITA	1	1
2	negrita con	TEXTO	1	3
3	*	CURSIVA	1	15
4	cursiva	TEXTO	1	16
5	*	CURSIVA	1	23
6	adentro	TEXTO	1	24
7	**	NEGRITA	1	32
8	(fin de línea)	FIN_LINEA	1	34
9	(fin de archivo)	EOF	2	1

Caso 11: Pestaña Tokens

Caso 12: Caso borde: # sin espacio no es encabezado

Archivo: `borde_sin_espacio.md`

Entrada

```
#sinespacio
```

Resultado esperado

Sin errores. Como no hay espacio después del #, es texto normal y no un encabezado.

Resultado obtenido

Tokens: TEXTO('#sinespacio'), FIN_LINEA, EOF. Sin errores. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado: **Correcto**

Captura



Caso 12: Pestaña Tokens

Caso 13: Caso borde: archivo vacío

Archivo: `borde_vacio.md`

Entrada

```
(archivo vacío)
```

Resultado esperado

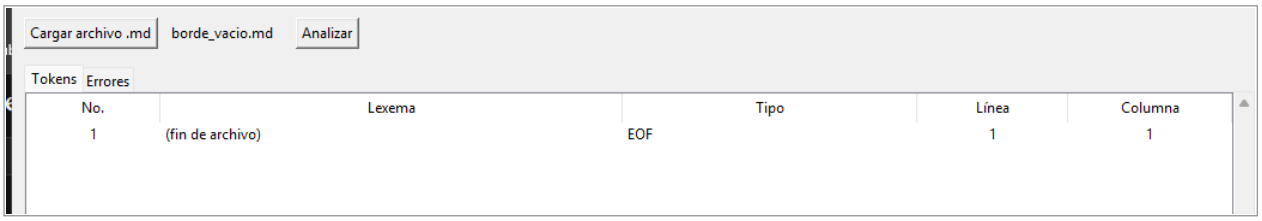
Sin errores y sin romperse. Solo debe salir el token EOF.

Resultado obtenido

Tokens: EOF. Sin errores. Los generadores terminaron sin excepción. Se generan los 3 reportes y el arbol.dot. Los botones se habilitan.

Resultado: **Correcto**

Captura



Caso 13: Pestaña Tokens